Python Programming - Lab Practice Set

50 problems: 17 Easy | 17 Medium | 16 Hard

Star patterns, logic building, scenario-based problems and LeetCode-style challenges covering Units I to IV

Instructions

- Read from standard input with `input()` and print to standard output with `print()`. Print **exactly** what is asked: no extra words, no extra spaces at the end of lines.
- Every problem lists the input format, output format, constraints and sample test cases. Always test your program on the samples first, then on your own edge cases (smallest value, largest value, empty or "none" cases).
- Easy = direct use of one or two concepts. Medium = combining concepts and careful logic. Hard = multi-step logic, 2D data, algorithms or classes.
- Problems marked **LeetCode-style** are adapted from well-known interview problems; the original problem number is shown in the topics line.
- Suggested approach: write the idea in 2-3 lines of pseudocode, trace it on the sample by hand, then code it.

Index of Problems

ID	Title	Level	Topics
E1	Number Pyramid	Easy	Patterns, Nested loops, Strings
E2	Alphabet Staircase	Easy	Patterns, ASCII / chr(), Loops
E3	Hollow Right Triangle	Easy	Patterns, Star Patterns, Nested loops
E4	Star Hourglass	Easy	Patterns, Star Patterns, Nested loops
E5	Town Electricity Bill	Easy	Conditionals, Scenario, Arithmetic
E6	Armstrong Number Hunt	Easy	Loops, Digits, Math logic
E7	Caesar Cipher with Digits	Easy	Strings, ASCII, Modulo
E8	Move Zeroes	Easy	Lists, Two pointers | LC 283
E9	Bracket Checker	Easy	Strings, Stack (list), Logic | LC 20
E10	Word Frequency Board	Easy	Dictionary, Strings, Sorting
E11	Nearest Point to Origin	Easy	Tuples, Lists, Functions
E12	Club Membership Report	Easy	Sets, Input handling
E13	Stairs with Broken Steps	Easy	Recursion / DP, Lists | LC 70
E14	Collatz Champion	Easy	Loops, While, Math logic
E15	Shop Receipt Formatter	Easy	String formatting, f-strings, Lists
E16	Roll Call Pairs	Easy	Tuples, zip(), Lists
E17	Scholarship Eligibility	Easy	Boolean operators, Conditionals, Scenario
M1	Hollow Diamond	Medium	Patterns, Star Patterns, Nested loops
M2	Spiral Matrix Builder	Medium	Patterns, 2D lists, Logic | LC 59
M3	Butterfly Pattern	Medium	Patterns, Star Patterns, Nested loops
M4	X Mark	Medium	Patterns, Star Patterns, 2D thinking
M5	Parking Lot Revenue	Medium	Strings, Time parsing, Scenario
M6	Library Fine Calculator	Medium	Dictionary, Strings, Scenario
M7	Weekly Attendance Report	Medium	Strings, Lists, Scenario
M8	Rank List with Ties	Medium	Sorting, Lambda, Scenario

ID	Title	Level	Topics
M9	Longest Unique Stretch	Medium	Strings, Sliding window, Dictionary | LC 3
M10	Prime Gap Finder	Medium	Math logic, Sieve, Lists
M11	Magic Square Inspector	Medium	2D lists, Logic, Functions
M12	Run-Length Compression	Medium	Strings, Loops, Logic | LC 443
M13	Password Strength Auditor	Medium	Strings, Functions, Scenario
M14	Spreadsheet Column Names	Medium	Math logic, Strings, Functions | LC 168/171
M15	Anagram Clusters	Medium	Strings, Dictionary, Sorting
M16	Friend Suggestions	Medium	Sets, Dictionary, Scenario
M17	Operation Pipeline	Medium	Functions as objects, Lambda, Dictionary
H1	Concentric Square	Hard	Patterns, Nested loops, Logic
H2	Centered Pascal Triangle	Hard	Patterns, Lists, Strings | LC 118
H3	Star Wave	Hard	Patterns, Star Patterns, Logic
H4	Hollow Hourglass	Hard	Patterns, Star Patterns, Nested loops
H5	Trapped Rain Water	Hard	Lists, Two pointers, Logic | LC 42
H6	Cinema Group Booking	Hard	2D lists, Scenario, Logic
H7	Merging Booked Slots	Hard	Sorting, Lists, Logic | LC 56
H8	Subarray Sum Detective	Hard	Lists, Dictionary, Prefix sums | LC 560
H9	Warmer Days Ahead	Hard	Lists, Stack, Logic | LC 739
H10	Coin Change Planner	Hard	Dynamic programming, Lists, Logic | LC 322/518
H11	League Table Builder	Hard	Strings, Dictionary, Sorting
H12	Word Hunt Grid	Hard	2D lists, Recursion, Backtracking | LC 79
H13	Mini Bank (Classes)	Hard	OOP / Classes, Exceptions, Dictionary
H14	Island Survey	Hard	2D lists, BFS / DFS, Logic | LC 200
H15	Sudoku Validator	Hard	2D lists, Sets, Logic | LC 36
H16	Conference Rooms Needed	Hard	Lists, Sorting, Scenario | LC 253

EASY LEVEL

Problem E1: Number Pyramid

EASY

Topics: Patterns, Nested loops, Strings

Print a pyramid of **N** rows. Row *i* (counting from 1) starts with **N - i** spaces, then the numbers 1, 2, ..., i in increasing order, followed by i-1, i-2, ..., 1 in decreasing order. There is no space between digits.

Input Format

The first line contains an integer N.

Output Format

Print the pyramid. Do not print trailing spaces.

Constraints

1 <= N <= 9

Sample Input 0	Sample Output 0

4	`   1`
	`  121`
	` 12321`
	`1234321`

Sample Input 1	Sample Output 1

6	`     1`
	`    121`
	`   12321`
	`  1234321`
	` 123454321`
	`12345654321`

Problem E2: Alphabet Staircase

EASY

Topics: Patterns, ASCII / chr(), Loops

Print a staircase of **N** rows made of capital letters. Row *i* has exactly *i* letters. The letters continue from where the previous row stopped, starting with A, and **wrap around to A** after Z.

For N = 4 the rows are A, BC, DEF, GHIJ.

Input Format

The first line contains an integer N.

Output Format

Print N lines, one row per line.

Constraints

1 <= N <= 10

Sample Input 0	Sample Output 0

4	`A`
	`BC`
	`DEF`
	`GHIJ`

Sample Input 1	Sample Output 1

8	`A`
	`BC`
	`DEF`
	`GHIJ`
	`KLMNO`
	`PQRSTU`
	`VWXYZAB`
	`CDEFGHIJ`

Problem E3: Hollow Right Triangle

EASY

Topics: Patterns, Star Patterns, Nested loops, Conditionals

Print a right-angled triangle of **N** rows using *****, but only its border: row 1 has one star, the last row (row N) is completely filled with N stars, and every row in between has a star at its first and last position with spaces in the middle. Do not print trailing spaces.

Input Format

The first line contains N.

Output Format

Print the triangle.

Constraints

1 <= N <= 20

Sample Input 0	Sample Output 0

5	`*`
	`**`
	`* *`
	`*  *`
	`*****`

Sample Input 1	Sample Output 1
2	`*` `**`

Sample Input 2	Sample Output 2
1	`*`

Problem E4: Star Hourglass

EASY

Topics: Patterns, Star Patterns, Nested loops, Logic

Print an hourglass made of ***** with **2N - 1** rows. The first row has 2N - 1 stars, every next row has two stars fewer and is shifted one space to the right so that the shape stays centred, until the middle row has a single star. After that the rows grow back in the same way. Do not print trailing spaces.

Input Format

The first line contains N.

Output Format

Print the hourglass.

Constraints

1 <= N <= 15

Sample Input 0	Sample Output 0

4	`*******`
	` *****`
	`  ***`
	`   *`
	`  ***`
	` *****`
	`*******`

Sample Input 1	Sample Output 1

2	`***`
	` *`
	`***`

Problem E5: Town Electricity Bill

EASY

Topics: Conditionals, Scenario, Arithmetic

An electricity board calculates a monthly bill from the customer category and the units used. Category **H** is household and **C** is commercial.

Household energy charge: first 100 units cost Rs 3 per unit, units 101 to 200 cost Rs 5 per unit, and every unit above 200 costs Rs 8.

Commercial energy charge: first 200 units cost Rs 9 per unit, every unit above 200 costs Rs 12.

If the energy charge is more than Rs 1000, a **surcharge** of 10% of the energy charge is added (use integer arithmetic: energy * 10 // 100).

A **fixed charge** is always added: Rs 50 for household and Rs 150 for commercial. Bill = energy charge + surcharge + fixed charge.

If the category is not H or C, or the units are negative, print INVALID.

Input Format

The first line contains a character, the category.

The second line contains an integer U, the units used.

Output Format

Print the bill amount as an integer, or INVALID.

Constraints

-50 <= U <= 2000

Sample Input 0	Sample Output 0
H 250	1370

Sample Input 1	Sample Output 1
C 150	1635

Problem E6: Armstrong Number Hunt

EASY

Topics: Loops, Digits, Math logic

A number is an **Armstrong number** if the sum of its digits, each raised to the power of the number of digits, equals the number itself. For example 153 = 1^3 + 5^3 + 3^3.

Given a range A to B (both included), find all Armstrong numbers in it.

Input Format

The first line contains A.

The second line contains B.

Output Format

Line 1: how many Armstrong numbers are in the range.

Line 2: the numbers in increasing order separated by single spaces, or -1 if there are none.

Constraints

1 <= A <= B <= 100000

Sample Input 0	Sample Output 0
100 999	4 153 370 371 407

Sample Input 1	Sample Output 1
10 99	0 -1

Problem E7: Caesar Cipher with Digits

EASY

Topics: Strings, ASCII, Modulo

A secret-message app shifts every character by K positions. Rules:

1) Letters rotate within the alphabet (A-Z or a-z) and keep their case. 2) Digits rotate within 0-9. 3) Any other character stays unchanged.

K can be negative, which means shifting backwards.

Input Format

The first line contains an integer K.

The second line contains the message (it may contain spaces and symbols).

Output Format

Print the encrypted message.

Constraints

-1000 <= K <= 1000

1 <= length of message <= 200

Sample Input 0	Sample Output 0
3 Hello, World 2026!	Khoor, Zruog 5359!

Sample Input 1	Sample Output 1
-5 Zebra-99	Uzwmv-44

Problem E8: Move Zeroes

EASY

Topics: Lists, Two pointers | LeetCode-style (LC 283)

Given a list of N integers, move all the zeroes to the end while keeping the relative order of the non-zero elements. **Challenge:** try to do it in one pass without creating a second list.

(Inspired by LeetCode 283)

Input Format

The first line contains N.

The second line contains N integers.

Output Format

Line 1: the rearranged list, space separated.

Line 2: the number of zeroes.

Constraints

1 <= N <= 1000

-1000 <= each element <= 1000

Sample Input 0	Sample Output 0
8 0 1 0 3 12 0 5 0	1 3 12 5 0 0 0 0 4

Sample Input 1	Sample Output 1
3 7 8 9	7 8 9 0

Problem E9: Bracket Checker

EASY

Topics: Strings, Stack (list), Logic | LeetCode-style (LC 20)

A code editor checks whether the brackets **( ) [ ] { }** in a line are balanced. All other characters are ignored.

If the line is balanced print VALID. Otherwise print INVALID followed by a space and a position (counting from 1): the position of the first closing bracket that does not match, or, if every closing bracket was fine but some opening brackets are never closed, the position of the **earliest** unclosed opening bracket.

(Inspired by LeetCode 20)

Input Format

One line containing the text.

Output Format

VALID, or INVALID k

Constraints

1 <= length <= 1000

Sample Input 0	Sample Output 0
`{[()]}`	`VALID`

Sample Input 1	Sample Output 1
`(a[b)c]`	`INVALID 5`

Sample Input 2	Sample Output 2
`({[]}`	`INVALID 1`

Problem E10: Word Frequency Board

EASY

Topics: Dictionary, Strings, Sorting

A blog wants to show how often each word appears. Words are separated by spaces, comparison is **case-insensitive**, and the punctuation characters . , ! ? are removed before counting.

Print every distinct word (in lowercase) with its count, ordered by count (highest first). Words with the same count are ordered alphabetically.

Input Format

One line of text.

Output Format

One line per distinct word: <word> <count>

Constraints

1 <= length of text <= 500

Sample Input 0	Sample Output 0

`The cat and the hat. The end, and THE start!`	`the 4`
	`and 2`
	`cat 1`
	`end 1`
	`hat 1`
	`start 1`

Sample Input 1	Sample Output 1
`go go GO now? now!`	`go 3` `now 2`

Problem E11: Nearest Point to Origin

EASY

Topics: Tuples, Lists, Functions

A drone logs N points (x, y) on a map. Find the point closest to the origin (0, 0); if several are equally close, choose the one that appeared first. Use squared distance, so no square roots are needed.

Also count how many points lie in each quadrant. Q1: x>0,y>0. Q2: x<0,y>0. Q3: x<0,y<0. Q4: x>0,y<0. Points lying on an axis belong to no quadrant.

Input Format

The first line contains N.

Each of the next N lines contains two integers x and y.

Output Format

Line 1: x y of the nearest point.

Line 2: its squared distance.

Line 3: four integers: counts in Q1 Q2 Q3 Q4.

Constraints

1 <= N <= 100

-1000 <= x, y <= 1000

Sample Input 0	Sample Output 0
5 3 4 -1 2 0 5 -2 -2 4 -1	-1 2 5 1 1 1 1

Sample Input 1	Sample Output 1
3 0 0 5 5 -5 -5	0 0 0 1 0 1 0

Problem E12: Club Membership Report

EASY

Topics: Sets, Input handling

Three college clubs - Coding, Music and Sports - each have a list of members. A student can be in more than one club. Find out how members are spread across the clubs.

Input Format

For each of the three clubs (in the order Coding, Music, Sports): one line with an integer K, followed by one line with K names separated by spaces. Names are single words and are unique within a club.

Output Format

Line 1: number of students in exactly one club.

Line 2: number of students in exactly two clubs.

Line 3: number of students in all three clubs.

Line 4: total number of distinct students.

Line 5: names of students in all three clubs, sorted alphabetically, or NONE.

Constraints

1 <= K <= 50

Sample Input 0	Sample Output 0
4 amit riya neha kabir 3 neha kabir sara 3 neha sara dev	3 2 1 6 neha

Sample Input 1	Sample Output 1
2 anu raj 2 raj anu 1 zoe	1 2 0 3 NONE

Problem E13: Stairs with Broken Steps

EASY

Topics: Recursion / DP, Lists | LeetCode-style (LC 70)

You are climbing a staircase with N steps. At each move you can climb 1 or 2 steps. Some steps are **broken** and you must never land on them (you may jump over them using a 2-step move). Count the number of different ways to reach the top (step N). Step 0 is the ground and is never broken.

(Inspired by LeetCode 70 and 746)

Input Format

Line 1: N.

Line 2: K, the number of broken steps.

Line 3: K integers, the broken step numbers.

Output Format

Print the number of ways.

Constraints

1 <= N <= 50

1 <= K <= N-1 (step N is never broken)

Sample Input 0	Sample Output 0
5 1 3	2

Sample Input 1	Sample Output 1
10 3 2 5 7	2

Problem E14: Collatz Champion

EASY

Topics: Loops, While, Math logic

The Collatz process on a number x: if x is even, replace it with x / 2; if x is odd, replace it with 3x + 1. Repeat until x becomes 1. The number of replacements made is the number of *steps* (so 1 needs 0 steps).

Given N, look at all starting numbers from 1 to N and find the one needing the **most steps** (the smallest such number if there is a tie).

Input Format

The first line contains N.

Output Format

Line 1: that starting number.

Line 2: its number of steps.

Line 3: the highest value reached in its sequence (including the start).

Constraints

1 <= N <= 10000

Sample Input 0	Sample Output 0
10	9 19 52

Sample Input 1	Sample Output 1
27	27 111 9232

Problem E15: Shop Receipt Formatter

EASY

Topics: String formatting, f-strings, Lists

A small shop prints receipts. For every item the receipt shows the name (left-aligned in 10 characters), the quantity (right-aligned in 4), the unit price (right-aligned in 8 with 2 decimals) and the amount (right-aligned in 9 with 2 decimals).

The receipt starts with a header line (ITEM QTY PRICE AMOUNT laid out in the same columns), then a line of 31 hyphens, the items, another line of 31 hyphens, and finally three lines: TOTAL, GST(5%) and PAYABLE. In each of these three lines the label is left-aligned in 22 characters and the value is right-aligned in 9 characters with 2 decimals. GST is 5% of the total.

Input Format

Line 1: N.

Next N lines: name quantity price (price has 2 decimals).

Output Format

Print the receipt exactly as described.

Constraints

1 <= N <= 20

item names are single words of at most 10 characters

Sample Input 0	Sample Output 0
3 pen 10 5.50 notebook 3 45.00 bag 1 810.00	ITEM QTY PRICE AMOUNT ------------------------------ pen 10 5.50 55.00 notebook 3 45.00 135.00 bag 1 810.00 810.00 ------------------------------ TOTAL 1000.00 GST(5%) 50.00 PAYABLE 1050.00

Sample Input 1	Sample Output 1
2 chalk 12 2.50 duster 2 10.00	ITEM QTY PRICE AMOUNT ------------------------------ chalk 12 2.50 30.00 duster 2 10.00 20.00 ------------------------------ TOTAL 50.00 GST(5%) 2.50 PAYABLE 52.50

Problem E16: Roll Call Pairs

EASY

Topics: Tuples, zip(), Lists

A teacher keeps student names and marks in two separate lists. Pair them up (the i-th name with the i-th mark) using tuples, then report: the topper (the first one in the list if several have the highest marks), the class average with 2 decimals, how many students scored **strictly above** the average, and their names in input order.

Input Format

Line 1: N.

Line 2: N names separated by spaces.

Line 3: N integer marks separated by spaces.

Output Format

Line 1: topper name and marks.

Line 2: average with 2 decimals.

Line 3: number of students above average.

Line 4: their names, or NONE.

Constraints

1 <= N <= 100

0 <= marks <= 100

Sample Input 0	Sample Output 0
4 riya aman neha kabir 85 90 85 70	aman 90 82.50 3 riya aman neha

Sample Input 1	Sample Output 1
3 zed amy bob 50 50 50	zed 50 50.00 0 NONE

Problem E17: Scholarship Eligibility

EASY

Topics: Boolean operators, Conditionals, Scenario

A college decides a scholarship from four values: marks percentage M, attendance percentage A, monthly family income I (in rupees) and B (1 if the student has any backlog, otherwise 0). Apply the rules in this order:

FULL: M >= 90 and A >= 75 and no backlog. **HALF**: M >= 75 and A >= 75 and no backlog and I <= 30000. **QUARTER**: M >= 60 and A >= 65 and I <= 15000 (a backlog is allowed). Otherwise **NONE**.

If M or A is outside 0 to 100, B is not 0 or 1, or I is negative, print INVALID.

Input Format

Four lines: M, A, I, B (one integer each).

Output Format

Print FULL, HALF, QUARTER, NONE or INVALID.

Constraints

-100 <= each value <= 1000000

Sample Input 0	Sample Output 0
92 80 50000 0	FULL

Sample Input 1	Sample Output 1
78 80 28000 0	HALF

Sample Input 2	Sample Output 2
62 66 12000 1	QUARTER

Sample Input 3	Sample Output 3
78 70 20000 0	NONE

MEDIUM LEVEL

Problem M1: Hollow Diamond — MEDIUM

Topics: Patterns, Star Patterns, Nested loops, Logic

Print a hollow diamond made of ***** whose widest row is in the middle. The diamond has **2N - 1** rows, the top row has a single star, and the middle row has its stars at the two ends of width 2N - 1. Only the border stars are printed; the inside is empty (spaces).

For N = 4 the diamond looks like the one in the sample. Do not print trailing spaces.

Input Format

The first line contains N.

Output Format

Print the diamond.

Constraints

1 <= N <= 15

Sample Input 0	Sample Output 0
4	<pre> * * * * * * * * * * * *</pre>

Sample Input 1	Sample Output 1
1	*

Sample Input 2	Sample Output 2
3	<pre> * * * * * * * *</pre>

Problem M2: Spiral Matrix Builder — MEDIUM

Topics: Patterns, 2D lists, Logic | LeetCode-style (LC 59)

Fill an N x N grid with the numbers 1, 2, ..., N*N in a **clockwise spiral** starting at the top-left corner and going right first. Print the grid. Every number must be right-aligned in a column whose width equals the number of digits of N*N, and columns are separated by a single space.

(Inspired by LeetCode 59)

Input Format

The first line contains N.

Output Format

Print the N x N grid.

Constraints

1 <= N <= 10

Sample Input 0	Sample Output 0

3	`1 2 3`
	`8 9 4`
	`7 6 5`

Sample Input 1	Sample Output 1

5	` 1  2  3  4  5`
	`16 17 18 19  6`
	`15 24 25 20  7`
	`14 23 22 21  8`
	`13 12 11 10  9`

Problem M3: Butterfly Pattern — MEDIUM

Topics: Patterns, Star Patterns, Nested loops, Logic

Print a butterfly of stars. For row i from 1 to N the row consists of i stars, then 2 * (N - i) spaces, then i stars. After row N, the rows N-1, N-2, ..., 1 follow in the same format, so the pattern has 2N - 1 rows in total and is symmetric top to bottom.

Input Format

The first line contains N.

Output Format

Print the butterfly.

Constraints

1 <= N <= 15

Sample Input 0	Sample Output 0

4	`*       *`
	`**     **`
	`***   ***`
	`********`
	`***   ***`
	`**     **`
	`*       *`

Sample Input 1	Sample Output 1
1	`**`

Sample Input 2	Sample Output 2

3	`*    *`
	`**  **`
	`******`
	`**  **`
	`*    *`

Problem M4: X Mark

MEDIUM

Topics: Patterns, Star Patterns, 2D thinking, Logic

Print the letter **X** made of stars in an N x N square, where N is odd. The stars are on the main diagonal and the anti-diagonal; all other cells are spaces. Do not print trailing spaces on any line.

Input Format

The first line contains N (odd).

Output Format

Print N lines.

Constraints

1 <= N <= 31, N is odd

Sample Input 0	Sample Output 0

5	`*   *`
	` * *`
	`  *`
	` * *`
	`*   *`

Sample Input 1	Sample Output 1

7	`*     *`
	` *   *`
	`  * *`
	`   *`
	`  * *`
	` *   *`
	`*     *`

Sample Input 2	Sample Output 2
1	*

Problem M5: Parking Lot Revenue

MEDIUM

Topics: Strings, Time parsing, Scenario

A parking lot records, for each vehicle, the plate number and the entry and exit times (HH:MM, 24-hour clock, same day). Fees are calculated as follows:

1) A stay of **10 minutes or less** is free. 2) Otherwise the time is rounded **up** to full hours; the first hour costs Rs 20 and every further hour costs Rs 15. 3) A vehicle is never charged more than Rs 150 for one visit.

Find the total revenue, the plate that paid the highest fee (the earliest one in the input if tied) and how many vehicles parked for free.

Input Format

Line 1: N, the number of vehicles.

Next N lines: PLATE ENTRY EXIT (for example MH12 10:15 12:20). Exit time is never earlier than entry time.

Output Format

Line 1: total revenue.

Line 2: the plate with the highest fee and the fee, separated by a space.

Line 3: number of free vehicles.

Constraints

1 <= N <= 100

Sample Input 0	Sample Output 0
5 KA01 09:00 09:08 MH12 10:15 12:20 DL08 08:00 17:30 UP32 14:00 14:10 GJ05 13:05 14:06	235 DL08 150 2

Sample Input 1	Sample Output 1
2 AB1 10:00 10:11 CD2 11:00 11:50	40 AB1 20 0

Problem M6: Library Fine Calculator — MEDIUM

Topics: Dictionary, Strings, Scenario

A library keeps a log of events. **ISSUE id day** means book *id* was lent on day number *day*. **RETURN id day** means it was returned on that day. A book can be issued again after it is returned.

A borrower may keep a book for **7 days** free. For every day beyond that the fine is **Rs 2 per day**. (A book issued on day 1 and returned on day 12 has been kept for 11 days, so 4 days late.)

Find the total fine collected and the books still not returned at the end of the log.

Input Format

Line 1: N, the number of events.

Next N lines: ISSUE id day or RETURN id day. The log is always valid and in order of day.

Output Format

Line 1: total fine.

Line 2: the number of books not returned.

Line 3: those book ids sorted alphabetically separated by spaces, or NONE.

Constraints

1 <= N <= 200

Sample Input 0	Sample Output 0
6 ISSUE B1 1 ISSUE B2 3 RETURN B1 12 ISSUE B3 5 RETURN B2 10 ISSUE B1 15	8 2 B1 B3

Sample Input 1	Sample Output 1
3 ISSUE X 1 RETURN X 31 ISSUE Y 40	46 1 Y

Problem M7: Weekly Attendance Report — MEDIUM

Topics: Strings, Lists, Scenario

A teacher records attendance for N students over D days as a string of P (present) and A (absent) characters. For each student print the attendance percentage with **one digit after the decimal point**.

Then list the **defaulters** (attendance strictly below 75%) in input order, and report the student with the longest run of **consecutive absences** (the first such student in the input if tied).

Input Format

Line 1: N and D.

Next N lines: a name and a string of length D (separated by a space).

Output Format

N lines: <name> <percentage>.

Then: Defaulters: <names separated by spaces> or Defaulters: NONE.

Then: Longest absence: <name> <days>, or Longest absence: NONE if nobody was ever absent.

Constraints

1 <= N <= 50

1 <= D <= 60

Sample Input 0	Sample Output 0

`4 6`
`riya PPAPPP`
`aman AAPPAA`
`neha PPPPPP`
`kabir PAAAPP`
	`riya 83.3`
	`aman 33.3`
	`neha 100.0`
	`kabir 50.0`
	`Defaulters: aman kabir`
	`Longest absence: kabir 3`

Sample Input 1	Sample Output 1

`2 4`
`sam PPPP`
`tia PPPP`
	`sam 100.0`
	`tia 100.0`
	`Defaulters: NONE`
	`Longest absence: NONE`

Problem M8: Rank List with Ties — MEDIUM

Topics: Sorting, Lambda, Scenario

After an exam, N students have marks. Create a rank list ordered by marks (highest first); students with equal marks are ordered alphabetically by name. Ranks use **competition ranking**: students with equal marks share the same rank and the next rank is skipped. For example marks 90, 85, 85, 70 get ranks 1, 2, 2, 4.

Input Format

Line 1: N.

Next N lines: name and integer marks.

Output Format

N lines: <rank> <name> <marks>

Constraints

1 <= N <= 100

0 <= marks <= 100

names are single words and unique

Sample Input 0	Sample Output 0

`5`
`riya 85`
`aman 90`
`neha 85`
`kabir 70`
`sara 90`
	`1 aman 90`
	`1 sara 90`
	`3 neha 85`
	`3 riya 85`
	`5 kabir 70`

Sample Input 1	Sample Output 1
3 zed 50 amy 50 bob 50	1 amy 50 1 bob 50 1 zed 50

Problem M9: Longest Unique Stretch — MEDIUM

Topics: Strings, Sliding window, Dictionary | LeetCode-style (LC 3)

Given a string S, find the longest substring (a continuous part) in which **no character repeats**. Print its length and the substring itself. If several substrings have the same maximum length, print the one that appears first.

(Inspired by LeetCode 3)

Input Format

One line containing S (no spaces).

Output Format

Line 1: the length.

Line 2: the substring.

Constraints

1 <= |S| <= 100000

Sample Input 0	Sample Output 0
abcabcbb	3 abc

Sample Input 1	Sample Output 1
pwwkew	3 wke

Problem M10: Prime Gap Finder — MEDIUM

Topics: Math logic, Sieve, Lists

Given a range A to B, find how many primes it contains. Also find the **largest gap** between two consecutive primes inside the range, and the first pair of consecutive primes that has this gap.

Input Format

Line 1: A.

Line 2: B.

Output Format

Line 1: the number of primes in [A, B].

Line 2: gap p q (the largest gap and the first pair), or -1 if the range has fewer than two primes.

Constraints

1 <= A <= B <= 1000000

Sample Input 0	Sample Output 0
10 50	11 6 23 29

Sample Input 1	Sample Output 1
90 96	0 -1

Problem M11: Magic Square Inspector — MEDIUM

Topics: 2D lists, Logic, Functions

A square N x N grid is a **magic square** if (1) it contains every number from 1 to N*N exactly once, (2) all rows have the same sum, (3) all columns have that same sum and (4) both diagonals have that same sum.

Print YES and the magic sum if the grid is a magic square. Otherwise print NO and, on the next line, the **first** rule that fails, checking in the order: DISTINCT, ROWS, COLUMNS, DIAGONALS.

Input Format

Line 1: N.

Next N lines: N integers each.

Output Format

YES S or NO followed by the name of the failed rule on the next line.

Constraints

2 <= N <= 10

Sample Input 0	Sample Output 0
3 2 7 6 9 5 1 4 3 8	YES 15

Sample Input 1	Sample Output 1
3 2 7 6 9 5 1 4 8 3	NO COLUMNS

Sample Input 2	Sample Output 2
3 1 1 1 1 1 1 1 1 1	NO DISTINCT

Problem M12: Run-Length Compression — MEDIUM

Topics: Strings, Loops, Logic | LeetCode-style (LC 443)

A file compressor replaces each run of equal consecutive characters by the character followed by the run length. A run of length 1 is written as the character alone. So aaabccdddd becomes a3bc2d4 (run lengths can have more than one digit, e.g. a12).

Print the compressed string only if it is **strictly shorter** than the original; otherwise print the original unchanged. On the second line print COMPRESSED or ORIGINAL.

(Inspired by LeetCode 443)

Input Format

One line containing a string of lowercase letters.

Output Format

Line 1: the resulting string.

Line 2: COMPRESSED or ORIGINAL.

Constraints

1 <= length <= 1000

Sample Input 0	Sample Output 0
aaabccdddd	a3bc2d4 COMPRESSED

Sample Input 1	Sample Output 1
abcd	abcd ORIGINAL

Sample Input 2	Sample Output 2
aaaaaaaaaaaab	a12b COMPRESSED

Problem M13: Password Strength Auditor — MEDIUM

Topics: Strings, Functions, Scenario

A website scores each password with 7 rules (one point per rule satisfied): R1 length at least 8. R2 has an uppercase letter. R3 has a lowercase letter. R4 has a digit. R5 has a special character from the set @ # $ % & * !. R6 has no character repeated 3 or more times in a row. R7 does not contain the word 'password' in any letter case.

Score 7 is STRONG, score 5 or 6 is MEDIUM, and score 4 or less is WEAK.

Input Format

Line 1: N.

Next N lines: one password each (no spaces).

Output Format

N lines: <CATEGORY> <score>.

Last line: STRONG=a MEDIUM=b WEAK=c

Constraints

1 <= N <= 100

1 <= length of password <= 50

Sample Input 0	Sample Output 0
4 Hello@2024 password1 AAAbbb111 Tr0ub4dor&3	STRONG 7 WEAK 4 MEDIUM 5 STRONG 7 STRONG=2 MEDIUM=1 WEAK=1

Sample Input 1	Sample Output 1
2 abc Xy9#Zk2!pq	WEAK 3 STRONG 7 STRONG=1 MEDIUM=0 WEAK=1

Problem M14: Spreadsheet Column Names — MEDIUM

Topics: Math logic, Strings, Functions | LeetCode-style (LC 168/171)

Spreadsheet columns are named A, B, ..., Z, AA, AB, ..., AZ, BA, ..., ZZ, AAA and so on. Given a column number N, print its name, and given a column name, print its number.

(Inspired by LeetCode 168 and 171)

Input Format

Line 1: an integer N.

Line 2: a column name T (capital letters).

Output Format

Line 1: the name of column N.

Line 2: the column number of T.

Constraints

1 <= N <= 2147483647

1 <= |T| <= 7

Sample Input 0	Sample Output 0
28 ZY	AB 701

Sample Input 1	Sample Output 1
703 FXSHRXW	AAA 2147483647

Sample Input 2	Sample Output 2
52 AZ	AZ 52

Problem M15: Anagram Clusters

MEDIUM

Topics: Strings, Dictionary, Sorting

Two words are **anagrams** if they contain the same letters in a different order (like listen and silent). Given N words, form clusters of words that are anagrams of each other. Only clusters with **at least 2 words** are reported.

Print the number of such clusters. Then print each cluster on its own line with its words sorted alphabetically. Clusters are ordered by size (largest first) and clusters of equal size by their first word.

Input Format

Line 1: N.

Line 2: N lowercase words separated by spaces.

Output Format

Line 1: number of clusters (with 2 or more words).

Then one line per cluster.

Constraints

1 <= N <= 1000

Sample Input 0	Sample Output 0
9 eat tea tan ate nat bat listen silent enlist	3 ate eat tea enlist listen silent nat tan

Sample Input 1	Sample Output 1
3 abc def ghi	0

Problem M16: Friend Suggestions MEDIUM

Topics: Sets, Dictionary, Scenario

A social app suggests new friends. For a target user, a suggestion is a person who is a friend of at least one of the target's friends, but is **not** the target and **not** already a friend of the target. The number of **mutual friends** is how many of the target's friends are also friends of the suggested person.

Print suggestions ordered by mutual friends (highest first) and then by name. Friendships are always symmetric in the input.

Input Format

Line 1: N, the number of users.

Next N lines: name K f1 f2 ... fK (the user's friends).

Last line: the target user's name.

Output Format

One line per suggestion: name mutual_count. Print NONE if there are no suggestions.

Constraints

1 <= N <= 100

Sample Input 0	Sample Output 0
5 amit 2 riya neha riya 2 amit sara neha 3 amit sara dev sara 2 riya neha dev 1 neha amit	sara 2 dev 1

Sample Input 1	Sample Output 1
2 a 1 b b 1 a a	NONE

Problem M17: Operation Pipeline — MEDIUM

Topics: Functions as objects, Lambda, Dictionary

A data tool applies a list of operations, in order, to every number of a list. Supported operations: **ADD x** (add x), **MUL x** (multiply by x), **MOD x** (remainder when divided by x), **SQUARE**, **NEG** (change sign) and **ABS** (absolute value).

Print the final numbers and their sum. **Hint:** store the operations as lambda functions in a dictionary.

Input Format

Line 1: N.

Line 2: N integers.

Line 3: K, the number of operations.

Next K lines: one operation each.

Output Format

Line 1: the resulting numbers.

Line 2: their sum.

Constraints

1 <= N, K <= 100

1 <= x <= 100

Sample Input 0	Sample Output 0
4 -3 5 0 2 4 ADD 1 MUL 3 ABS MOD 5	1 3 3 4 11

Sample Input 1	Sample Output 1
3 1 -2 3 2 NEG SQUARE	1 4 9 14

HARD LEVEL

Problem H1: Concentric Square

HARD

Topics: Patterns, Nested loops, Logic

For a given N, print a square of size (2N - 1) x (2N - 1) made of concentric rings of digits. The outermost ring is N, the next ring is N-1, and so on, down to 1 in the centre. There are no spaces between the digits.

Input Format

The first line contains N.

Output Format

Print the square.

Constraints

1 <= N <= 9

Sample Input 0	Sample Output 0
4	4444444 4333334 4322234 4321234 4322234 4333334 4444444

Sample Input 1	Sample Output 1
2	222 212 222

Problem H2: Centered Pascal Triangle

HARD

Topics: Patterns, Lists, Strings | LeetCode-style (LC 118)

Print the first N rows of Pascal's triangle. Each row is written as its numbers separated by single spaces. Let W be the length of the **last** row as a string. Every row is printed with exactly (W - L) // 2 leading spaces, where L is the length of that row's string. Do not print trailing spaces.

(Inspired by LeetCode 118)

Input Format

The first line contains N.

Output Format

Print N lines.

Constraints

1 <= N <= 12

Sample Input 0	Sample Output 0
5	1 1 1 1 2 1 1 3 3 1 1 4 6 4 1

Sample Input 1	Sample Output 1
7	1 1 1 1 2 1 1 3 3 1 1 4 6 4 1 1 5 10 10 5 1 1 6 15 20 15 6 1

Problem H3: Star Wave HARD

Topics: Patterns, Star Patterns, Logic, Modulo

Print a wave of stars that is **N** rows tall and **W** columns wide. The wave starts at the top-left corner, goes down one row per column until it reaches the bottom row (row N), then goes up one row per column until it reaches the top row, and keeps repeating. In every column exactly one star is printed. Do not print trailing spaces.

Input Format

Line 1: N.

Line 2: W.

Output Format

Print N lines.

Constraints

2 <= N <= 10

1 <= W <= 60

Sample Input 0	Sample Output 0
3 9	* * * * * * * * *

Sample Input 1	Sample Output 1
4 14	* * * * * * * * * * * * * *

Sample Input 2	Sample Output 2
2 6	* * * * * *

Problem H4: Hollow Hourglass HARD

Topics: Patterns, Star Patterns, Nested loops, Logic

Print a **hollow** hourglass of height 2N - 1. The first and last rows are completely filled with 2N - 1 stars. The rows between them show only the two border stars (the slanted sides), moving inwards until the middle row, which has a single star, and then moving outwards again. Do not print trailing spaces.

Input Format

The first line contains N.

Output Format

Print the hourglass.

Constraints

1 <= N <= 15

Sample Input 0	Sample Output 0
4	******* * * * * * * * * * *******

Sample Input 1	Sample Output 1
2	*** * ***

Sample Input 2	Sample Output 2
1	*

Problem H5: Trapped Rain Water HARD

Topics: Lists, Two pointers, Logic | LeetCode-style (LC 42)

After rain, water collects between bars of different heights. Each bar has width 1. Given the heights of N bars, find the **total** amount of water trapped and the **maximum depth** of water above any single bar.

(Inspired by LeetCode 42)

Input Format

Line 1: N.

Line 2: N non-negative integers (heights).

Output Format

Line 1: total water.

Line 2: maximum depth above one bar (0 if no water).

Constraints

1 <= N <= 100000

0 <= height <= 100000

Sample Input 0	Sample Output 0
12 0 1 0 2 1 0 1 3 2 1 2 1	6 2

Sample Input 1	Sample Output 1
6 4 2 0 3 2 5	9 4

Problem H6: Cinema Group Booking HARD

Topics: 2D lists, Scenario, Logic

A cinema hall has R rows with C seats each, all initially free. Booking requests arrive one by one; request i asks for K_i **adjacent** seats in the same row.

For every request choose the row with the **smallest row number** that has K adjacent free seats, and inside that row the **leftmost** such block. Mark them booked. If no row can satisfy the request, it is rejected and nothing changes.

Input Format

Line 1: R and C.

Line 2: Q, the number of requests.

Next Q lines: one integer K each.

Output Format

For each request print row start-end (1-based), for example 2 1-4, or REJECTED.

Then print the total number of seats booked.

Then print the number of completely full rows.

Constraints

1 <= R, C <= 20

1 <= Q <= 100

1 <= K <= C

Sample Input 0	Sample Output 0
3 5 6 3 4 2 5 3 1	1 1-3 2 1-4 1 4-5 3 1-5 REJECTED 2 5-5 15 3

Sample Input 1	Sample Output 1
2 4 4 2 2 3 1	1 1-2 1 3-4 2 1-3 2 4-4 8 2

Problem H7: Merging Booked Slots

HARD

Topics: Sorting, Lists, Logic | LeetCode-style (LC 56)

A hall receives N booking requests, each as a time interval [a, b]. Overlapping or touching intervals (when the next start is less than or equal to the current end) are merged into one block. Print the merged blocks in increasing order and the total length of time covered (the sum of b - a over merged blocks).

(Inspired by LeetCode 56)

Input Format

Line 1: N.

Next N lines: a b with a less than or equal to b.

Output Format

Line 1: M, the number of merged blocks.

Next M lines: a b.

Last line: the total covered length.

Constraints

1 <= N <= 10000

0 <= a <= b <= 1000000

Sample Input 0	Sample Output 0
6 1 3 2 6 8 10 15 18 17 20 6 7	3 1 7 8 10 15 20 13

Sample Input 1	Sample Output 1
3 5 5 1 2 3 4	3 1 2 3 4 5 5 2

Problem H8: Subarray Sum Detective HARD

Topics: Lists, Dictionary, Prefix sums | LeetCode-style (LC 560)

Given N integers (they may be negative) and a target K, count the continuous subarrays whose sum is exactly K, and also report the length of the **longest** such subarray (0 if there is none).

(Inspired by LeetCode 560)

Input Format

Line 1: N and K.

Line 2: N integers.

Output Format

Line 1: the number of subarrays with sum K.

Line 2: the length of the longest one.

Constraints

1 <= N <= 100000

-1000 <= element <= 1000

-10^9 <= K <= 10^9

Sample Input 0	Sample Output 0
8 7 3 4 7 2 -3 1 4 2	4 4

Sample Input 1	Sample Output 1
4 0 1 -1 1 -1	4 4

Problem H9: Warmer Days Ahead HARD

Topics: Lists, Stack, Logic | LeetCode-style (LC 739)

For each day you are given the temperature. For every day, find how many days you have to wait until a **warmer** temperature arrives. If there is no future warmer day, the answer for that day is 0.

Also report the longest wait and the (first) day on which it occurs, or 0 -1 if every answer is 0.

(Inspired by LeetCode 739)

Input Format

Line 1: N.

Line 2: N integers, the temperatures.

Output Format

Line 1: the N waiting times.

Line 2: the longest wait and its day number (1-based).

Constraints

1 <= N <= 100000

-50 <= temperature <= 60

Sample Input 0	Sample Output 0
8 73 74 75 71 69 72 76 73	1 1 4 2 1 1 0 0 4 3

Sample Input 1	Sample Output 1
4 30 40 50 60	1 1 1 0 1 1

Sample Input 2	Sample Output 2
3 50 40 30	0 0 0 0 -1

Problem H10: Coin Change Planner HARD

Topics: Dynamic programming, Lists, Logic | LeetCode-style (LC 322/518)

A shop has coins of N different denominations (unlimited supply of each). To pay exactly T rupees find (a) the **minimum number of coins** needed, or -1 if it is impossible, and (b) the **number of different combinations** of coins that add up to T (the order of coins does not matter).

(Inspired by LeetCode 322 and 518)

Input Format

Line 1: N and T.

Line 2: N distinct positive integers.

Output Format

Line 1: minimum coins or -1.

Line 2: number of combinations.

Constraints

1 <= N <= 10

1 <= T <= 10000

1 <= coin <= 10000

Sample Input 0	Sample Output 0
3 11 1 2 5	3 11

Sample Input 1	Sample Output 1
1 3 2	-1 0

Sample Input 2	Sample Output 2
3 6 1 3 4	2 4

Problem H11: League Table Builder HARD

Topics: Strings, Dictionary, Sorting

A football league records match results as **TeamA goalsA - goalsB TeamB**, for example Tigers 2 - 1 Lions. A win gives 3 points, a draw 1 point and a loss 0 points. Build the final table sorted by: points (high first), then goal difference (high first), then goals scored (high first), then team name (alphabetical).

Team names are single words.

Input Format

Line 1: M, the number of matches.

Next M lines: one result each.

Output Format

One line per team, in table order: position team played won drawn lost GF GA GD points (all separated by single spaces).

Constraints

1 <= M <= 100

Sample Input 0	Sample Output 0
5 Tigers 2 - 1 Lions Lions 0 - 0 Bears Bears 3 - 3 Tigers Tigers 1 - 0 Bears Lions 2 - 2 Wolves	1 Tigers 3 2 1 0 6 4 2 7 2 Bears 3 0 2 1 3 4 -1 2 3 Lions 3 0 2 1 3 4 -1 2 4 Wolves 1 0 1 0 2 2 0 1

Sample Input 1	Sample Output 1
2 A 1 - 0 B B 1 - 0 A	1 A 2 1 0 1 1 1 0 3 2 B 2 1 0 1 1 1 0 3

Problem H12: Word Hunt Grid HARD

Topics: 2D lists, Recursion, Backtracking | LeetCode-style (LC 79)

You are given an R x C grid of lowercase letters and a word. The word can be formed by starting at a cell and moving to **horizontally or vertically adjacent** cells, using each cell at most once in one path.

Print YES followed by the number of different starting cells from which the word can be formed, or NO if the word cannot be formed at all.

(Inspired by LeetCode 79)

Input Format

Line 1: R and C.

Next R lines: a string of C letters each.

Last line: the word.

Output Format

YES k or NO

Constraints

1 <= R, C <= 6

1 <= length of word <= 15

Sample Input 0	Sample Output 0
3 4 abce sfcs adee abcced	YES 1

Sample Input 1	Sample Output 1
3 4 abce sfcs adee see	YES 1

Sample Input 2	Sample Output 2
3 4 abce sfcs adee abcb	NO

Problem H13: Mini Bank (Classes) HARD

Topics: OOP / Classes, Exceptions, Dictionary, Scenario

Simulate a small bank. You **must use a class** for accounts. Savings accounts must always keep a **minimum balance of Rs 500**. Commands:

CREATE name amount - open an account; amount must be at least 500. **DEPOSIT name amount** - amount must be greater than 0. **WITHDRAW name amount** - amount must be greater than 0 and the balance after withdrawal must stay at least 500. **TRANSFER from to amount** - same rules as a withdrawal from *from*, then a deposit to *to*.

A failed command changes nothing and prints **ERROR k REASON**, where k is the command number (starting at 1). Check in this order: DUPLICATE (CREATE of an existing name), NO_ACCOUNT (any named account missing), INVALID_AMOUNT, INSUFFICIENT.

Input Format

Line 1: N, the number of commands.

Next N lines: the commands.

Output Format

First the ERROR lines in order of occurrence. After that print every account as name balance, sorted by name.

Constraints

1 <= N <= 200

names are single words, amounts are integers

Sample Input 0	Sample Output 0
8 CREATE ravi 1000 CREATE meena 700 DEPOSIT ravi 500 WITHDRAW meena 300 TRANSFER ravi meena 800 WITHDRAW anil 100 CREATE ravi 900 DEPOSIT meena -5	ERROR 4 INSUFFICIENT ERROR 6 NO_ACCOUNT ERROR 7 DUPLICATE ERROR 8 INVALID_AMOUNT meena 1500 ravi 700

Sample Input 1	Sample Output 1
3 CREATE a 400 CREATE a 600 WITHDRAW a 100	ERROR 1 INVALID_AMOUNT a 500

Problem H14: Island Survey HARD

Topics: 2D lists, BFS / DFS, Logic | LeetCode-style (LC 200)

A satellite map is an R x C grid where 1 is land and 0 is water. An **island** is a group of land cells connected horizontally or vertically. Find the number of islands, the area (number of cells) of the largest island, and how many islands touch the border of the map (have at least one cell in the first or last row or column).

(Inspired by LeetCode 200 and 695)

Input Format

Line 1: R and C.

Next R lines: a string of C characters, each 0 or 1.

Output Format

Line 1: number of islands.

Line 2: area of the largest island (0 if none).

Line 3: number of islands touching the border.

Constraints

1 <= R, C <= 50

Sample Input 0	Sample Output 0
4 5 11000 11000 00100 00011	3 4 2

Sample Input 1	Sample Output 1
3 3 000 000 000	0 0 0

Sample Input 2	Sample Output 2
3 4 1111 1001 1111	1 10 1

Problem H15: Sudoku Validator HARD

Topics: 2D lists, Sets, Logic | LeetCode-style (LC 36)

A partially filled 9 x 9 Sudoku board uses digits 1 to 9 and the character **.** for an empty cell. The board is **valid** if no digit repeats within any row, any column or any of the nine 3 x 3 boxes (empty cells are ignored). The boxes are numbered 1 to 9 row by row from the top-left.

Print VALID, or INVALID followed by the first broken unit, checking all rows (1 to 9) first, then all columns (1 to 9), then all boxes (1 to 9). For example INVALID ROW 3.

(Inspired by LeetCode 36)

Input Format

9 lines, each with 9 characters (digits 1-9 or .).

Output Format

VALID or INVALID UNIT k

Constraints

The board has exactly 9 rows and 9 columns.

Sample Input 0	Sample Output 0
53..7.... 6..195... .98....6. 8...6...3 4..8.3..1 7...2...6 .6....28. ...419..5 ....8..79	VALID

Sample Input 1	Sample Output 1
53..7..5. 6..195... .98....6. 8...6...3 4..8.3..1 7...2...6 .6....28. ...419..5 ....8..79	INVALID ROW 1

Sample Input 2	Sample Output 2
53..7.... 6..195... .98....6. 8...6...3 4..8.3..1 7...2...6 .6....28. ...419..5 5...8..79	INVALID COL 1

Problem H16: Conference Rooms Needed HARD

Topics: Lists, Sorting, Scenario | LeetCode-style (LC 253)

A company has N meetings, each with a start and an end time (in minutes). A room that becomes free at minute t can be used for a meeting that starts at minute t. Find the **minimum number of rooms** needed so that no two overlapping meetings share a room, and the **earliest minute** at which this maximum number of simultaneous meetings is reached.

(Inspired by LeetCode 253)

Input Format

Line 1: N.

Next N lines: start end (start is less than end).

Output Format

Line 1: minimum rooms.

Line 2: the earliest minute when that many meetings are running together.

Constraints

1 <= N <= 10000

0 <= start < end <= 100000

Sample Input 0	Sample Output 0
5 0 30 5 10 15 20 10 25 12 18	4 15

Sample Input 1	Sample Output 1
2 1 5 5 9	1 1